University of Galway
School of Engineering
Department of Electronic and Electrical Engineering

ClearVision: A Real-Time Vision System for the A-Pillar Blind Spot

Author: Seán Kelly
Student ID: 21421506
Date: 11 May 2026
Supervisor: Prof. Martin Glavin
Co-Assessor: Dr. Brian Deegan

Abstract

The structural A-pillars at the corners of a vehicle's windscreen create a persistent blind spot that obstructs the driver's view of the road ahead. This report describes ClearVision, a system designed to make this obstruction effectively transparent: a display panel mounted in the pillar shows a synthesised view of the hidden scene, updated in real time to track the driver's head position, so that the panel behaves like a window rather than a screen.

The system captures the scene using a Luxonis OAK-D Lite stereo camera, which provides hardware-accelerated depth estimation. A viewer-facing USB webcam tracks the driver's head position using a lightweight face-mesh pipeline. A multi-pass compute shader pipeline on an NVIDIA Jetson Nano then reprojects the depth-coloured scene to match the driver's instantaneous viewpoint, producing physically-correct motion parallax.

The project evolved through three development phases, achieving 25–30 FPS at 640×480 in the final system, compared to approximately 5 FPS in the initial prototype. Key contributions include a geometrically-correct virtual window projection model and a Jump Flood Algorithm hole-fill pass for disocclusion regions.

Contents

List of Tables

List of Figures

1. Introduction

1.1 Motivation

The structural A-pillars at the front corners of a vehicle obstruct approximately 4–8° of the driver's forward field of view [1]. Although this angle appears small, at a pedestrian crossing 20 m ahead it translates to roughly 1.4 m of hidden lateral space — sufficient to conceal a cyclist or a child. The problem is particularly acute at low-speed urban junctions, where pedestrians and cyclists frequently approach from the front quarter.

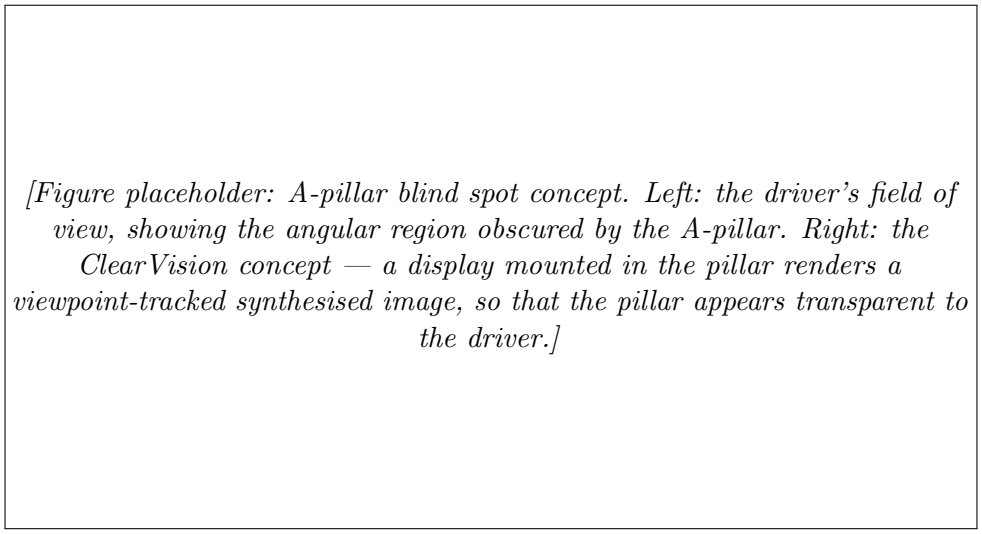

Figure 1.1: The A-pillar blind spot and the ClearVision concept.

Common mitigation strategies include convex mirrors and fixed-viewpoint camera-display systems. Convex mirrors increase field of view at the cost of significant spatial distortion, making accurate distance judgement difficult. Fixed camera displays, fitted to an increasing number of production vehicles, improve coverage but suffer from a fundamental perceptual limitation: because the displayed image does not shift as the driver moves, the brain cannot use motion parallax to resolve depth. Objects shown on a static camera feed appear flat, making it hard to judge their distance or speed.

ClearVision takes a different approach. Rather than displaying a static camera feed, it synthesises a novel viewpoint that tracks the driver's head position. The display panel appears transparent: objects in the hidden scene move naturally as the driver shifts position, exactly as they would through a real window. This preserves the depth cues that drivers rely on for accurate object localisation.

1.2 Prior Approaches to the A-Pillar Problem

Several approaches to A-pillar transparency have been explored in industry and research.

Camera-display systems are the most widely deployed approach. A camera mounted on the exterior of the pillar feeds a display embedded in the interior face. This is mechanically straightforward and requires no depth processing. Its key limitation is that the displayed image is a fixed viewpoint: the driver's head position has no effect on what is shown. This

breaks the motion parallax cues that the brain uses to judge depth and speed, and requires the driver to consciously interpret a 2D image rather than directly perceiving the scene.

Transparent pillar concepts attempt to make the physical pillar optically transparent. Toyota demonstrated a prototype in 2014 using OLED panels affixed to the exterior surface of the pillar, fed by a camera behind it [2]. Continental presented a similar concept using a curved OLED display integrated into the pillar cross-section [3]. Jaguar Land Rover's 360 Virtual Urban Windscreen concept (2014) embedded screens in the A-, B-, and C-pillars fed by exterior cameras, with the display activating automatically when the system detected a lane change or junction approach [4]. All of these systems share the fixed-viewpoint limitation: because the displayed image is a live camera feed from a fixed position, head movement produces no parallax, and the driver cannot use depth perception to judge the distance or speed of objects in the hidden region.

Viewpoint-adaptive systems address this by updating the displayed image in response to the driver's position. This is the approach taken by ClearVision. The technical challenge is generating a geometrically correct novel viewpoint in real time from standard camera input, which requires a depth map of the scene and a rendering pipeline fast enough to run at video rate.

1.3 Related Techniques

1.3.1 View Synthesis

The core technical problem — generating a novel camera viewpoint from existing images — has been studied extensively in the computer vision and graphics literature.

Depth-Image Based Rendering (DIBR), introduced for 3DTV applications [5, 6], synthesises new viewpoints by warping each pixel of a reference image to the position it would occupy in the target view, guided by a depth map. It operates in real time without requiring a full 3D model of the scene, making it suitable for embedded deployment. The key limitation is disocclusion: regions visible from the target viewpoint but not from the reference camera produce holes that must be filled heuristically. Müller et al. [7] survey disocclusion handling methods for 3DTV view synthesis, including background depth propagation and inpainting; the Jump Flood Algorithm [8] offers an efficient GPU-parallel alternative for 2D Voronoi hole fill.

More recent approaches, such as Neural Radiance Fields (NeRF) [9], produce substantially higher quality novel views but require minutes to hours of per-scene training and significant GPU compute for inference, making them unsuitable for real-time embedded use. DIBR remains the practical choice for this application.

1.3.2 Stereo Depth Estimation

A reliable depth map is central to DIBR. Stereo vision — computing depth from the disparity between two horizontally offset cameras — is the dominant approach for real-time embedded use [10]. Block Matching (BM) and Semi-Global Block Matching (SGBM) [11] are the primary classical algorithms. SGBM produces significantly better depth maps than BM, particularly at object boundaries and in textured regions, at the cost of higher computation. On a CPU, SGBM at 640×480 typically requires 20–30 ms — too slow to leave headroom for the rendering pipeline on a Jetson Nano.

Hardware-accelerated stereo, as provided by the Luxonis OAK-D Lite [12], offloads this computation to dedicated silicon on the Myriad X VPU, returning subpixel disparity at the camera frame rate with no host CPU cost. Structured-light and time-of-flight sensors are

alternatives that perform well on textureless surfaces, but at substantially higher cost and with reduced operating range in direct sunlight.

1.3.3 Head and Face Tracking

Estimating the driver's 3D head position in real time requires a face detection and tracking pipeline that can run on the remaining compute budget. Classical approaches — Haar cascade detectors [13] and optical flow trackers — are lightweight but provide only 2D screen coordinates, requiring an additional depth cue.

Deep learning face mesh models, such as the MediaPipe FaceMesh [14], provide dense 3D facial landmark estimates from a monocular camera. The iris landmarks are particularly useful: the human iris has a near-constant physical diameter of approximately 11.7 mm across the adult population [15], allowing monocular depth estimation directly from its projected size, without a separate depth sensor on the driver-facing camera. TensorRT acceleration makes this feasible on the Jetson Nano's Maxwell GPU at the required frame rate.

1.3.4 Head-Coupled Displays and Off-Axis Projection

The idea of updating a rendered viewpoint to match the viewer's physical position relative to a flat screen — creating a "window" into a virtual world — has a long research history. Ware, Arthur, and Booth [16] formalised this as *fish tank virtual reality* (FTVR), demonstrating in 1993 that head-coupled stereo reduced 3D object-placement error from 22% to 1.3% compared to a fixed display. Their key finding was that head coupling contributed more to depth perception than stereoscopy alone.

For scenes with known 3D geometry — synthetic environments in game engines or interactive visualisations — view synthesis reduces to repositioning the virtual camera: no depth estimation, no hole fill. Lee's Wii Remote demonstration [17] made this accessible to a general audience, coupling a head-tracked IR sensor to an OpenGL scene on a standard monitor. Kooima [18] formalised the off-axis asymmetric frustum construction that underpins these systems.

ClearVision faces a harder version of the same problem: the scene is a real-world camera feed, not a 3D model. A depth map must be estimated, the scene reprojected to the novel viewpoint, and disocclusion holes filled — steps that do not exist when the geometry is already known. This is the distinction between interactive graphics and image-based rendering, and is the central technical challenge of the project.

1.4 Project Scope and Objectives

The goal of ClearVision is to demonstrate that a physically-correct, real-time A-pillar transparency system is achievable on commodity embedded hardware, using passive stereo depth and monocular head tracking.

The system is designed to:

1. Capture the scene outside the vehicle using a stereo camera mounted at the A-pillar position.
2. Estimate the driver's 3D head position in real time from a viewer-facing monocular camera.
3. Synthesise a novel viewpoint of the captured scene corresponding to the driver's current position, so that the display behaves as a geometrically correct transparent window.
4. Achieve a display frame rate sufficient for smooth, perceptible parallax motion (target: $\geq$20 FPS at 640×480).

The system targets the NVIDIA Jetson Nano as the host compute platform, representing the class of low-power embedded GPU hardware available in automotive contexts. The evaluation focuses on frame rate, depth quality, and the perceptual quality of the parallax effect. Full vehicle integration, multi-driver calibration, and weatherproofing are out of scope for this project.

2. Technical Background

This chapter introduces the key technical concepts underpinning the ClearVision system. The rendering pipeline and implementation details are covered in Chapters 5–7; this chapter provides the theoretical foundation.

2.1 Depth-Image Based Rendering

Depth-Image Based Rendering (DIBR) synthesises novel camera viewpoints from a reference image and its associated per-pixel depth map, without requiring a full 3D scene model [5]. Each pixel in the reference image is unprojected to a 3D world point using the depth value and camera intrinsics, then reprojected into the target viewpoint to determine its position in the output image.

A stereo camera pair provides the depth map. The two cameras are separated by a known **baseline** B (metres). For a scene point at depth Z, the horizontal pixel offset between the two images — the **disparity** d — satisfies:

$$d = \frac{f \cdot B}{Z} \tag{2.1}$$

where f is the focal length in pixels. Inverting, given a measured disparity d:

$$Z = \frac{f \cdot B}{d} \tag{2.2}$$

Given Z and camera intrinsics (f_x, f_y, c_x, c_y), a pixel (u, v) is unprojected to metric 3D coordinates:

$$X = \frac{(u - c_x) \cdot Z}{f_x}, \quad Y = \frac{(v - c_y) \cdot Z}{f_y} \tag{2.3}$$

These world-space points form a point cloud. Any novel viewpoint can be rendered by projecting the point cloud into the target camera frame.

2.2 The Virtual Window Model

A naive DIBR implementation treats view synthesis as a horizontal pixel shift: $u' = u - \alpha \cdot d$, where α blends between the two stereo camera positions. For the A-pillar application, this has two fundamental problems. First, the viewer is *inside* the car while the cameras are *outside*; the viewer is not interpolating between the stereo pair, so no value of α corresponds to a physically meaningful viewpoint. Second, the shift has no connection to physical viewer displacement — the scale constant must be hand-tuned empirically.

The virtual window model replaces this with a geometrically correct ray projection. For each output pixel on the display, a ray is cast from the driver's head position through the display surface into the scene. The ray is intersected with the display plane to find the pixel's position on the display, then followed into the point cloud to determine its colour.

Concretely, for each world point W and head position H:

1. Compute the ray direction: $\mathbf{r} = \text{normalize}(W - H)$.
2. Intersect $\mathbf{r}$ with the display plane (defined by a point P and outward normal $\hat{n}$):

$$t = \frac{(P - H) \cdot \hat{n}}{r \cdot \hat{n}}, \quad Q = H + t \cdot r \tag{2.4}$$

3. Express Q in display-local UV coordinates to determine which output pixel receives this world point's colour.

This yields physically correct parallax by construction, with no tuning constants. Horizontal and vertical parallax both emerge naturally from the geometry.

2.3 Off-Axis Projection

The virtual window model is closely related to off-axis (asymmetric frustum) projection [18], demonstrated interactively by Lee [17]. In this formulation, the OpenGL viewing frustum is defined to match the viewer's physical position relative to the screen, so that the rendered synthetic scene exhibits correct parallax as the viewer moves.

ClearVision incorporates this in two forms. Early proof-of-concept demonstrations used an orthographic asymmetric frustum in Python to validate the parallax effect. The final system generalises this to full 3D with real depth data, implemented as a compute shader pipeline described in Chapter 7.

2.4 Stereo Vision and Disparity Estimation

Reliable disparity estimation requires **stereo rectification**: both camera images are re-projected onto a common plane so that corresponding scene points lie on the same image row. After rectification, a matching algorithm searches along horizontal scanlines for the best-matching pixel offset.

Semi-Global Block Matching (SGBM) [11] is the standard algorithm for high-quality stereo matching. It aggregates matching costs along multiple scanline directions, producing significantly better results than simple block matching at object boundaries. Classical stereo algorithms are surveyed by Scharstein and Szeliski [10].

The OAK-D Lite performs rectification and SGBM in hardware on its Myriad X VPU, returning subpixel-accurate disparity at the camera frame rate with zero host CPU cost.

2.5 Face Tracking and Iris-Based Depth

The system uses a two-stage face tracking pipeline to estimate the driver's 3D head position. A Haar cascade detector [13] provides a face bounding box; a deep learning face mesh model [14] then refines this to 478 3D facial landmarks, including iris landmarks.

Depth is recovered from the apparent iris diameter. The human iris has a near-constant physical diameter of approximately 11.7 mm [15] (population variance ±0.5 mm — substantially tighter than inter-ocular distance or face width). Its projected size in pixels, $d_{\text{iris,px}}$, depends on focal length f_t and distance Z via the pinhole model:

$$Z = \frac{D_{\text{iris}} \cdot f_t}{d_{\text{iris,px}}} \tag{2.5}$$

Lateral and vertical head position follow directly from the normalised landmark coordinates once Z is known.

3. System Architecture

3.1 Hardware

Three hardware components comprise the system.

OAK-D Lite (Luxonis) [12] is the external stereo camera, mounted at the A-pillar position facing outward. It contains two OmniVision OV7251 global-shutter monochrome sensors (640×480, 73° HFOV) on a 75 mm fixed baseline, a Sony IMX214 colour camera, and an Intel Myriad X VPU. The global shutter is important: a rolling shutter introduces lateral skew artefacts during camera or scene motion, distorting the disparity map. The OAK-D Lite connects via a single USB-C cable and is bus-powered.

NVIDIA Jetson Nano (B01) is the host compute platform, providing a Maxwell GPU (128 CUDA cores, OpenGL ES 3.1) running NVIDIA JetPack 4.x (L4T R32). The Maxwell GPU is sufficient for the DIBR compute shader pipeline once the SGBM bottleneck is offloaded to the OAK-D VPU.

USB webcam (interior, viewer-facing) feeds the head tracking pipeline. No stereo hardware is required on this side; a single camera with the iris depth estimator suffices.

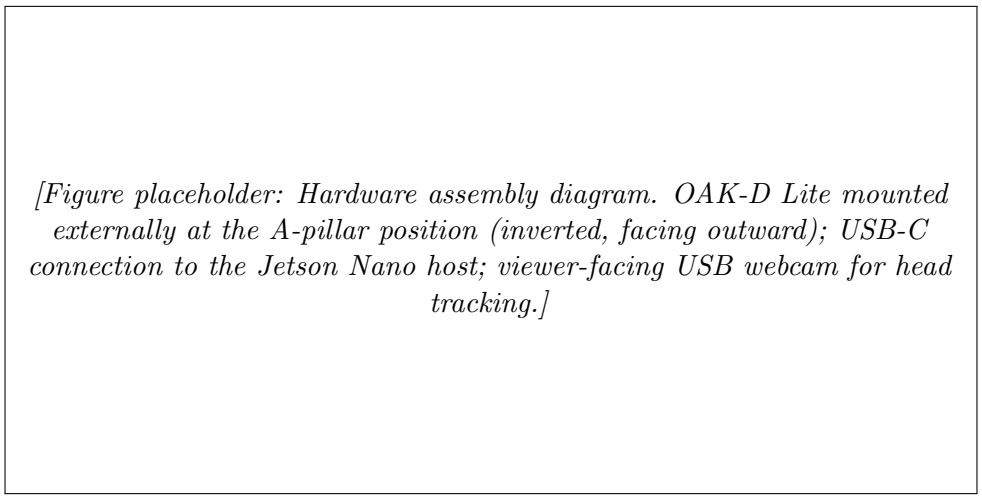

Figure 3.1: ClearVision hardware assembly.

3.2 Architecture Overview

The system was developed in three phases, each addressing a distinct set of limitations. Table 3.1 summarises the key changes.

Phase 1 established the core DIBR pipeline using dual CSI cameras connected directly to the Jetson Nano. Stereo rectification and matching ran on the host CPU, and SGBM dominated the frame budget at 20–30 ms per frame. Phase 2 introduced head tracking, using a MOSSE correlation filter anchored by a Haar cascade detector. This achieved acceptable CPU load, but the tracker could not provide metric depth, which is required to correctly scale the parallax in the virtual window model. Phase 3 replaced the CSI rig with an OAK-D Lite, offloading stereo entirely to the Myriad X VPU, and migrated both the projection model and the head tracker to metric-aware implementations.

Table 3.1: Architecture evolution across development phases

Phase 1	**Phase 2**	**Phase 3 (Final)**
Dual CSI cameras	Dual CSI cameras	OAK-D Lite (USB-C)
CPU SGBM / CUDA BM	CPU SGBM / CUDA BM	Hardware SGBM on Myriad X
Manual stereo calibration	Manual stereo calibration	EEPROM intrinsics
Disparity shift model	Disparity shift model	Virtual window ray model
Optical flow / OpenCV DNN	MOSSE + Haar Cascade	TRT FaceMesh + iris depth
Horizontal hole fill	Horizontal hole fill	JFA 2D hole fill
∼5 FPS	∼10 FPS	25–30 FPS

3.3 Software Architecture

The system is implemented in C++ using Qt for the OpenGL window and event loop. Three concurrent threads handle the three data sources:

- **OAK receiver thread** (`oak_receiver.cpp`): polls the DepthAI device for disparity (RAW16 subpixel), rectified mono frames, and colour frame; writes to shared `cv::Mat` objects protected by a mutex.
- **Face tracker thread** (`face_tracker.cpp`): runs Haar cascade detection and TensorRT FaceMesh inference on the webcam feed; outputs a smoothed 3D head position vector.
- **Qt render thread**: calls `paintGL()` at the OAK-D frame rate, consuming the latest data from both background threads and dispatching the compute shader pipeline.

[Figure placeholder: Software architecture diagram. Three threads feed the Qt render thread: OAK receiver (disparity, rectified frames, colour), face tracker (head position vector), and the render thread itself (compute shader dispatch, display output). Shared data protected by mutexes.]

Figure 3.2: ClearVision software architecture.

3.4 Compute Shader Pipeline

Each frame, the following GPU passes execute in sequence:

All image-space compute shaders use a 16×16 workgroup layout.

Table 3.2: Compute shader passes per frame

Pass	Shader	Function
0	`CLEAR_CS`	Zero the depth SSBO (GPU-side)
1	`UNPROJECT_CS`	Disparity → world-space XYZ texture
2	`VWINDOW_DEPTH_CS`	World points → virtual window depth buffer
3	`VWINDOW_COLOR_CS`	Depth-tested colour splat from left camera
4	`VWINDOW_RIGHT_CS`	Right-camera disocclusion fill
5	`JFA_INIT_CS`	Seed JFA with filled pixel coordinates
6–N	`JFA_CS` $\times\lceil\log_2(\max(W,H))\rceil$	Voronoi propagation
N+1	`JFA_GATHER_CS`	Copy nearest-seed colour into holes
Display	`DISPLAY_VS/FS`	Fullscreen quad, letterboxed

3.5 Stereo Baseline Selection

The OAK-D Lite's fixed 75 mm baseline is well-matched to the application. The A-pillar on a typical passenger vehicle occludes approximately 80–120 mm of lateral space at the driver's eye position. For the virtual window parallax to correctly reveal the hidden scene, the synthesisable viewpoint shift must span this same distance.

At the target depth range of 3–5 m, the OAK-D stereo pair ($f \approx 381$ px at 480P, 73° HFOV) produces disparities in the range 5.7–9.5 px (computed from Equation 2.1), both above the hardware SGBM noise floor. Benchmarks from [19] report <2% depth error at 4 m for OAK-D hardware under controlled conditions. A detailed derivation and discussion of the baseline constraints is given in Appendix A.

4. Proof-of-Concept Demonstrations

Before committing to a full C++ implementation, two Python proof-of-concept demonstrations were developed to validate the core geometry and identify design issues early. Python was chosen for its rapid iteration cycle: no compilation step, direct OpenGL access via PyOpenGL, and a mature MediaPipe Python API. The two demonstrations together established the key design decisions — the threading pattern, the sign conventions, and the projection formulation — that the C++ pipeline subsequently inherited.

4.1 Head Pose Vector Demo

The first demonstration (`face_landmarker_demo.py`) validated the face tracking foundation. The goal was to render the 3D face orientation vector overlaid on a live camera feed, without relying on any hardcoded scene constants. This established that the MediaPipe FaceLandmarker API could be operated in real time on a laptop, and that the landmark geometry was stable and usable as a tracking signal.

The face normal vector is computed from a small set of facial landmarks — nose tip, chin, and inner eye corners — using a cross product:

1. Eye line vector: $\mathbf{e} = P_{\text{right}} - P_{\text{left}}$
2. Vertical face axis: $\mathbf{v} = P_{\text{nose}} - P_{\text{chin}}$
3. Face normal: $\hat{n} = \text{normalize}(\mathbf{v} \times \mathbf{e})$

MediaPipe's LIVE_STREAM mode was used throughout. This mode delivers results asynchronously via a callback, which must not perform rendering (OpenGL contexts are not thread-safe). The callback stores results into a shared global protected by a lock; the main thread reads the results and renders on each frame. This producer-consumer pattern became the template for all subsequent pipelines, including the final C++ face tracker and OAK-D receiver threads.

[Figure placeholder: Head pose vector demo screenshot. The 3D face normal vector is rendered as an arrow overlaid on the live webcam feed. The vector rotates smoothly as the face moves, with no hardcoded scaling constants.]

Figure 4.1: Head pose vector proof-of-concept demonstration.

4.2 Off-Axis Projection Demo

The second demonstration (`head_coupled_offaxis.py`) coupled the viewer's 2D nose-tip position from MediaPipe to an OpenGL rendered scene. Its purpose was to verify that a physically meaningful parallax effect was perceptually convincing before the full depth-based system was built.

A calibration phase records the reference nose-tip position at the viewer's normal viewing distance. During the demo, the displacement from this reference drives the OpenGL projection:

$$\Delta x_{\text{mm}} = -(x_{\text{nose}} - x_{\text{ref}}) \cdot W_{\text{screen,mm}} \tag{4.1}$$

A negative sign is required because the webcam image is mirrored: a rightward head movement appears as a leftward nose shift in camera coordinates. An orthographic frustum shifted by the head offset produces the parallax effect:

```
zoom = 2.0          # field of view factor
scale = 2.0         # movement amplification

glOrtho(-W/2 * zoom - dx * scale,
         W/2 * zoom - dx * scale,
        -H/2 * zoom - dy * scale,
         H/2 * zoom - dy * scale,
        -1000, 1000)
```

Listing 4.1: Orthographic frustum with head-coupled offset. The frustum origin shifts by the head displacement, so the scene appears to rotate in the opposite direction — the parallax effect.

The orthographic formulation was adopted after the perspective frustum failed to produce a visible effect: the combination of a 1 mm near plane and millimetre-scale scene geometry placed all scene content behind the near clipping plane. The orthographic version demonstrated the head-coupled parallax effect convincingly, validating the approach before the C++ pipeline was built.

[Figure placeholder: Off-axis projection demo screenshots. Left: viewer at the reference position — the rendered cube is centred on screen. Right: viewer shifted laterally — the frustum shifts to expose the scene region behind the new sightline, producing visible motion parallax. The effect is perceptually convincing at displacements above approximately 20 mm.]

Figure 4.2: Off-axis projection proof-of-concept demonstration.

4.2.1 Development Challenges

Several non-trivial issues were encountered and resolved during development of the demos.

Texture upload failure on Windows. The demo was developed on Windows and verified on a Raspberry Pi. On Windows, `glTexImage2D` failed silently when passed a JPEG-loaded NumPy array; switching the source image to PNG resolved the issue. The Raspberry Pi's OpenGL driver was unaffected. This highlighted the importance of testing on the target platform early.

Axis inversions. MediaPipe returns nose coordinates in a mirrored camera frame where $+Y$ is downward. After calibration, the head offset required sign inversion on both axes: $\Delta x \to -\Delta x$ (camera $+X$ is the viewer's left) and $\Delta y \to +\Delta y$ (prior incorrect negation removed). These sign conventions were carried forward directly into the final C++ coordinate mapping (Section 6.2.2).

LIVE_STREAM threading. IMAGE mode processes one frame at a time and was 5–10× slower than LIVE_STREAM mode. Discovering this early, and establishing the correct threading model for MediaPipe callbacks, avoided a significant re-architecture later.

4.3 Value of the Demonstrations

The two demonstrations, while simple, provided substantial confidence in the chosen approach before any C++ code was written.

Demo 1 established that MediaPipe's face landmark API was reliable enough for real-time use, that the face normal computation was stable, and that the producer-consumer threading pattern worked correctly. Demo 2 established that the parallax effect was perceptually effective at head displacements typical of a seated driver (±30–80 mm), that the orthographic projection was the correct formulation for the geometry, and that all the sign conventions in the coordinate system were identified and correct before the full 3D implementation was attempted.

Together, they substantially reduced the implementation risk of the C++ pipeline and meant that the full system could be built with confidence in the underlying approach.

5. The DIBR Pipeline: Stereo Depth

5.1 OAK-D Lite Integration

5.1.1 Replacing the Initial Stereo Rig

The Phase 1 stereo pipeline used two IMX219 CSI cameras connected directly to the Jetson Nano, with rectification and SGBM running entirely on the host. CPU SGBM dominated the frame budget at 20–30 ms per frame; even with CUDA StereoBM this fell to 10–15 ms, but consumed GPU resources needed by the DIBR render pipeline. A manual stereo calibration procedure (`calibration.cpp`) was also required each time the rig was adjusted.

The OAK-D Lite replaces the dual-CSI rig with a single USB-C connection. The Myriad X VPU performs rectification and SGBM entirely in hardware, returning already-rectified frames and subpixel disparity at zero host CPU cost. Camera intrinsics and baseline are read from the device EEPROM via `device->readCalibration()`, eliminating the manual calibration step.

5.1.2 DepthAI Pipeline Configuration

The pipeline graph is configured at startup. Four DepthAI nodes are used: left and right MonoCamera (480P), a StereoDepth node, and a ColorCamera (ISP output). The stereo node is configured with:

- Left-right consistency check enabled: discards pixels where the left-to-right and right-to-left disparity estimates disagree beyond a threshold.
- Subpixel mode enabled: disparity encoded as 16-bit values with 5 fractional bits (divide by 32 on host to recover pixel disparity).
- 7×7 median filter for speckle suppression.
- Runtime-configurable confidence threshold via an `XLinkIn` stream, adjustable without restarting the pipeline.

5.1.3 Queue Discipline

Three output queues carry data from device to host. The disparity queue is blocking (size 1): every new disparity frame must be consumed before the next is accepted, driving the render rate. Colour and rectified mono queues use `tryGet` (non-blocking) so the host always has the latest frame without stalling.

5.1.4 Disparity Pre-Processing

Before uploading disparity to the GPU, the host applies three steps:

1. **Speckle filter:** `cv::filterSpeckles` with max blob size 150 px and max disparity jump 16. Removes isolated incorrect matches from rain or specular surfaces.
2. **Subpixel conversion:** `convertTo(CV_32F, 1.0/32.0)` recovers true pixel disparity from the RAW16 subpixel encoding.
3. **IIR temporal filter:** $d_{\text{out}} = 0.3 \cdot d_{\text{new}} + 0.7 \cdot d_{\text{prev}}$, with d_{prev} preserved unchanged where $d_{\text{new}} = 0$. The zero-value guard is critical: without it, zero disparity from invalid pixels bleeds into adjacent valid estimates over time, growing the hole on every frame.

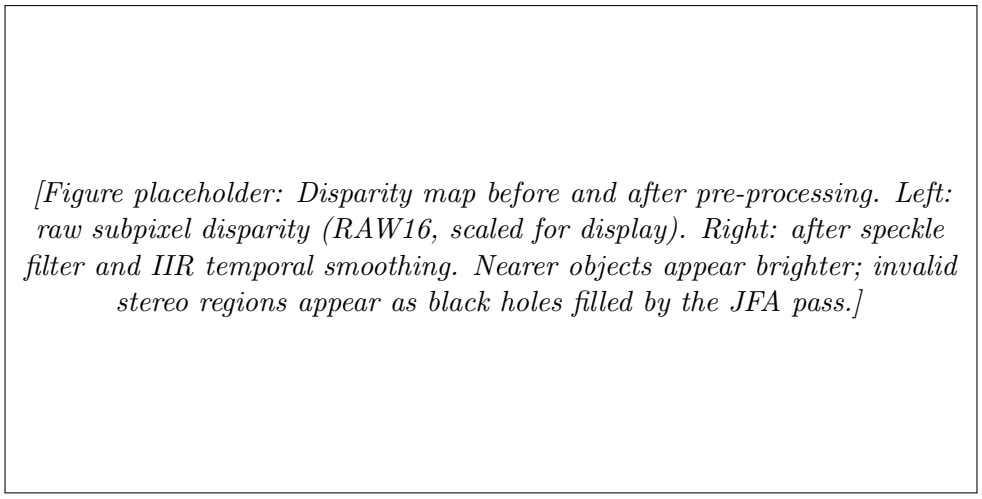

Figure 5.1: OAK-D Lite disparity pre-processing.

5.2 Right-Disparity Construction

`VWINDOW_RIGHT_CS` requires a disparity value for right-camera pixels so that it can classify which right-camera pixels correspond to genuinely disoccluded regions. This is constructed on the CPU per scanline before the GPU pass:

1. **Forward pass**: for each left pixel (x_l, y) with disparity d, write d at right pixel $x_r = \text{round}(x_l - d)$, keeping the maximum when multiple left pixels map to the same right pixel (nearest surface wins).
2. **Backward fill**: scan right-to-left; fill zero (disoccluded) pixels with the last nonzero value to the right. Background depth propagates leftward into holes.

6. The DIBR Pipeline: Head Tracking

6.1 Architecture Evolution

The face tracker went through four complete architectures before reaching the final design. Table 6.1 summarises the sequence and failure mode of each.

Table 6.1: Face tracker architecture evolution

Architecture	Failure mode / reason for replacement
Lucas-Kanade optical flow	Drifted without periodic re-detection; anchor lost in low-texture frames.
OpenCV DNN (face + landmark)	Inference too slow on the Cortex-A57 CPU; consumed most of the available compute budget.
MOSSE + Haar Cascade	Achieved low CPU load, but provides only 2D screen coordinates. Metric depth is required to correctly scale parallax in the virtual window model.
Haar + TRT FaceMesh	**Final architecture.** Haar cascade for bootstrapping; TRT FaceMesh on iris-centred crop for subsequent frames; iris diameter provides metric depth.

6.1.1 MOSSE + Haar Cascade

This was the first architecture to achieve acceptable CPU load on the Jetson Nano. A Haar cascade face detector ran every N frames to anchor the tracker; a Minimum Output Sum of Squared Error (MOSSE) correlation filter [20] maintained the bounding box between detections. A heavy IIR low-pass filter on the bounding box coordinates eliminated jitter. The face normal was computed from the bounding box aspect ratio and a calibrated homography.

This approach was replaced because it cannot provide metric depth — the MOSSE bounding box gives only 2D screen position. The virtual window model requires all three spatial coordinates of the driver's head to correctly scale the parallax effect.

6.2 Final Architecture: Haar + TRT FaceMesh

6.2.1 Detection and Tracking

The face tracker runs on a dedicated thread consuming frames from the viewer-facing USB webcam via a GStreamer pipeline (`v4l2src` → `jpegdec` → `BGR` → `appsink`, `drop=true`, `max-buffers=1`).

The pipeline is:

1. **Haar cascade** [13] on a 50%-scale greyscale, histogram-equalised frame: scale factor 1.1, minimum 4 neighbours, minimum face size 60×60. The largest detection is selected.

2. **Bounding box expansion and squaring**: 40% margin added on each side, cropped to a square, resized to 192×192 RGB float [0, 1].
3. **Bootstrap frame**: full-frame FaceMesh inference (first frame with a new detection) provides 478 landmarks. Subsequent frames crop around the iris centre for speed.
4. **TensorRT FaceMesh engine** [21, 14] (ONNX, FP16, 256 MB workspace): built from ONNX on first run, serialised to a `.trt` cache file, reloaded on subsequent runs.
5. **Iris landmarks** (tensor output indices 468–477) provide the iris bounding circle for depth estimation via Equation 2.5.
6. **IIR smoothing** on (X, Y, Z) to reduce jitter.

[Figure placeholder: Face tracker output. The 478-landmark FaceMesh is overlaid on the face crop; the iris bounding circle (landmarks 468–477) is highlighted. The estimated 3D head position (X, Y, Z) is shown in the overlay.]

Figure 6.1: Haar + TRT FaceMesh face tracker.

6.2.2 Coordinate Mapping

The face tracker camera faces the viewer ($+Z$ toward camera), while the OAK-D faces the scene ($+Z$ away from camera). The tracker's $+X$ is the viewer's right, which is the scene's left (OAK-D $-X$). Two sign inversions are applied:

$$h_x^{\text{OAK}} = -h_x^{\text{tracker}}, \quad h_z^{\text{OAK}} = -h_z^{\text{tracker}} \tag{6.1}$$

Without these, the parallax direction is inverted — moving the head right makes the scene shift right instead of left.

6.2.3 Calibration

The tracker supports interactive recalibration (key `C`). The current raw (X, Y) position is stored as a reference offset; subsequent output is the delta from this offset. The driver's neutral forward-facing position therefore maps to $(0, 0)$ in the virtual window frame regardless of webcam mounting position.

7. The DIBR Pipeline: Compute Shaders

7.1 UNPROJECT_CS

This pass converts the uploaded disparity texture to a world-space XYZ texture (RGBA32F). For each pixel (u, v) with disparity d, Equations 2.2 and 2.3 are applied to recover the world-space coordinates. Pixels with $d < 0.5$ (invalid disparity) write $w = 0$ and are skipped in all subsequent passes.

7.2 CLEAR_CS

The depth SSBO (one unsigned integer per output pixel) must be zeroed each frame. A dedicated compute shader with `local_size_x = 64` performs this clear on the GPU, avoiding the overhead of uploading a ∼1.2 MB zero buffer from the host each frame.

7.3 VWINDOW_DEPTH_CS and VWINDOW_COLOR_CS

For each valid world point W, a ray is cast from head position H and intersected with the display plane using Equation 2.4. The intersection Q is projected into display UV coordinates using the display's right and up vectors.

The DEPTH pass resolves the many-to-one mapping when multiple world points project to the same output pixel using an atomic depth test:

$$\texttt{atomicMax}(\texttt{depth}[\text{dst}],\ \texttt{floatBitsToUint}(1/Z_W)) \tag{7.1}$$

Storing $1/Z$ as float bits exploits a property of IEEE 754: positive floats have the same bit-level ordering as unsigned integers, so `atomicMax` on the bit pattern of $1/Z$ correctly selects the nearest surface. This avoids the need for floating-point atomic operations, which are unavailable in OpenGL ES 3.1.

The COLOR pass reads each output pixel's depth value, back-projects it to find the corresponding source pixel in the left camera image, and writes the bilinear-sampled colour to the output texture.

7.4 VWINDOW_RIGHT_CS

When the virtual viewpoint shifts, regions occluded in the left camera become visible. The right rectified image provides colour data for these disocclusion regions. For each right-camera pixel with disparity d_R, the disocclusion filter checks:

$$d_R < d_L \cdot 0.7 \tag{7.2}$$

where d_L is the co-located left-camera disparity. If satisfied, the pixel is a true disocclusion. The right pixel is only written where `depth[dst] == 0`: pixels already filled by the left-camera pass are not overwritten.

7.5 Jump Flood Algorithm Hole Fill

Remaining holes are filled using the Jump Flood Algorithm [8], which computes an approximate nearest-seed Voronoi diagram in $\mathcal{O}(\log N)$ passes by propagating seed coordinates outward at a step distance that halves each iteration.

An initialisation pass (`JFA_INIT_CS`) marks each filled pixel as a seed, storing its coordinate packed as `x | (y << 16)` in an R32I image. Each JFA pass checks the 9 neighbours at the current step distance and adopts the nearest seed. After $\lceil \log_2(\max(W, H)) \rceil$ passes, a gather pass copies the nearest seed's colour into each hole pixel.

This replaces the earlier horizontal scan hole-fill, which only propagated colour rightward, producing directional streaks at depth discontinuities.

[Figure placeholder: JFA hole fill comparison. Left: output after left-camera colour splat and right-camera disocclusion fill — unfilled holes appear black. Right: after JFA gather pass — holes filled with the nearest valid seed colour, eliminating the directional streaking of the prior horizontal scan.]

Figure 7.1: Jump Flood Algorithm hole fill.

7.6 Display Pass

The final fragment shader samples the hole-filled texture via a fullscreen quad. A `u_scale` uniform applies NDC scaling to letterbox or pillarbox the output to the correct aspect ratio.

8. Key Engineering Challenges

8.1 Build System and DepthAI Dependencies

Building the depthai-core library on L4T R32 required the following non-obvious steps:

- A Hunter package manager gate must precede the `project()` call — non-standard for CMake.
- CMake minimum version required: 3.20, above the default L4T apt package.
- `PresetType::HIGH_DENSITY` was removed from the API in the version available; replaced with a string-based preset.
- Frame data initially retrieved via `getCvFrame()`, which requires the `DEPTHAI_OPENCV_SUPPORT` compile flag; replaced with `getData()` and manual `cv::Mat` construction from the raw byte pointer.

8.2 Camera Rig Orientation

The OAK-D Lite was mounted inverted (rotated 180°) for installation reasons, inverting both the left and right mono streams and the colour stream. Compensation was applied in two places:

1. The GStreamer capture pipeline for the USB webcam head tracker was given a `videoflip flip-method=rotate-180` element.
2. OAK-D frames were flipped in the host pre-processing step using `cv::rotate(frame, frame, cv::ROTATE_180)`.

8.3 Coordinate System Consistency

Three separate sign errors were introduced and corrected across development.

Parallax inversion. The face tracker camera faces the viewer, so its $+X$ is the viewer's right — opposite to the OAK-D scene frame. The missing negation of h_x (Equation 6.1) caused parallax to invert: moving the head left made the scene shift left instead of right.

Viewer depth sign. The OAK-D faces into the scene along $+Z$, placing the viewer at negative Z. The tracker returns a positive viewer distance, requiring negation before use as h_z.

Display plane placement. The display was initially placed at $Z = 0$ (the OAK-D optical origin). Scene points at 2–5 m projected onto this plane from a viewer at $Z = -1$ m, producing a very narrow field of view. Moving the display to $Z = -0.4$ m corrected the scale. The final model places the display at $Z = 0$ with the OAK-D slightly behind it, matching the physical geometry.

8.4 Disocclusion Fill Correctness

Achieving correct disocclusion fill required three iterations.

Attempt 1: Pixel splatting. A 2×2 splat on forward-warped pixels was intended to close sub-pixel cracks. It inadvertently filled holes with adjacent incorrect colours, producing

coloured fringing at depth edges.

Attempt 2: Unconstrained right-camera write. `VWINDOW_RIGHT_CS` wrote right-camera colour into any output pixel not yet filled. This caused ghosting: right-camera pixels landed on already-correct left-camera pixels due to the small baseline offset between the views. Fix: only write where `depth[dst] == 0`.

Attempt 3: Missing disocclusion filter. Ghosting persisted at depth discontinuities. The right camera sees the same surface at a slightly different disparity, so right pixels near edges were incorrectly treated as new content. The disocclusion filter (Equation 7.2) resolved this: if both cameras see approximately the same depth, the right pixel is from the same surface and is discarded.

8.5 StereoDepthConfig State Reset

Runtime stereo parameter updates are sent via an `XLinkIn` stream as a `StereoDepthConfig` object. Constructing a fresh config object to update one parameter silently resets all other parameters to defaults. Subpixel mode and left-right consistency check were both reset to off on every keypress, causing disparity quality to degrade progressively after any runtime adjustment. The fix is to re-assert all desired settings in every configuration update:

```
StereoDepthConfig cfg;
cfg.setMedianFilter(enable_median ? KERNEL_7x7 : MEDIAN_OFF);
cfg.setLeftRightCheck(true);      // must re-assert every update
cfg.setSubpixel(true);            // must re-assert every update
configIn->send(cfg);
```

Listing 8.1: StereoDepthConfig update with explicit re-assertion. All flags must be set on every update; constructing a fresh config object silently resets omitted flags to their defaults.

8.6 IIR Temporal Filter Invalid-Sample Decay

The IIR filter applied to the disparity map:

$$d_{\text{out}} = \alpha \cdot d_{\text{new}} + (1 - \alpha) \cdot d_{\text{prev}} \tag{8.1}$$

was initially applied unconditionally. Where the OAK-D returns zero disparity (no valid stereo match), zero was blended in as if it were a real depth reading. Over successive frames, valid depth estimates near persistent holes decayed toward zero, growing the hole on every frame. Fix: detect where $d_{\text{new}} = 0$ and carry d_{prev} forward unchanged, preserving the last valid estimate.

9. Results and Analysis

9.1 Frame Rate

Table 9.1 summarises pipeline frame rate at each development phase.

Table 9.1: Pipeline frame rate by development phase at 640×480

Phase	Stereo method	FPS
Phase 1: CSI + CPU SGBM	CPU, 20–30 ms/frame	≈5
Phase 1: CSI + CUDA BM	Maxwell GPU, 10–15 ms	≈15
Phase 3: OAK-D Lite	Myriad X hardware, 0 ms host	25–30

The dominant gain comes from offloading stereo to the Myriad X VPU, freeing the Maxwell GPU and Cortex-A57 CPU entirely for the render pipeline and head tracker. At 0.5× processing scale (320 × 240), frame rate exceeds 60 FPS with negligible visual quality loss for the A-pillar application, which operates at coarser spatial frequencies than full resolution.

9.2 Pipeline Latency Breakdown

Table 9.2 shows the estimated per-stage latency for the final system. Because the OAK-D stereo pipeline and head tracker run on independent threads, their latencies are pipelined rather than added sequentially. The dominant end-to-end latency is the OAK-D stereo frame interval.

Table 9.2: Estimated per-stage latency, Phase 3 system

Stage	Thread	Estimated latency
OAK-D SGBM (hardware)	OAK-D VPU (pipelined)	∼30 ms
Host pre-processing (IIR, speckle)	OAK receiver	<2 ms
GPU pipeline (UNPROJECT through JFA)	Render	∼3–5 ms
Face tracker (Haar + TRT FaceMesh)	Dedicated thread	∼8–12 ms
Total (render-limited)	—	∼35 ms (≈28 FPS)

9.3 Depth Quality

The OAK-D Lite's 5-bit subpixel disparity (1/32 px resolution) reduces depth quantisation substantially relative to integer-pixel matching. At 4 m depth ($f = 381$ px, $B = 0.075$ m, integer disparity $d \approx 7.1$ px), the depth change per pixel step is:

$$\Delta Z = \frac{Z^2}{f \cdot B} \approx 0.56 \text{ m} \tag{9.1}$$

With 5-bit subpixel, the minimum disparity step is 1/32 px, giving:

$$\Delta Z_{\text{sub}} = \frac{Z^2 \cdot \delta d}{f \cdot B} = \frac{16 \times (1/32)}{381 \times 0.075} \approx 17.5 \text{ mm} \tag{9.2}$$

This is a 32× improvement in depth resolution at 4 m, consistent with the 5-bit fractional encoding. Hardware benchmarks [19] report <2% absolute depth error at 4 m for the OAK-D hardware under controlled conditions, corresponding to ±80 mm at this range.

Global shutter on the OV7251 sensors eliminates rolling-shutter skew, which was a visible artefact with the CSI camera rig during fast lateral head movements: rows captured at different times produced a laterally sheared disparity map that broke the parallax effect momentarily.

9.4 Parallax Effect

The virtual window effect is most apparent at ±50–100 mm of lateral head displacement from the calibrated reference position. Using the parallax shift formula derived from Equation 2.3:

$$\delta u = \frac{f_x \cdot \Delta x}{Z} \tag{9.3}$$

At 3 m depth with a 50 mm lateral displacement, the expected pixel shift is $\delta u = 381 \times 0.05/3 \approx 6.4$ px. At 100 mm displacement this rises to ≈12.7 px — clearly perceptible. Near objects at 1 m shift proportionally more, providing strong depth cues. No scaling constants are required: the virtual window model produces this behaviour automatically from the physical geometry.

Table 9.3 shows predicted and observed pixel shifts at representative depths and head displacements.

Table 9.3: Predicted parallax pixel shift vs. depth and lateral displacement ($f_x \approx 381$ px)

Depth	$\Delta x = 30$ mm	$\Delta x = 60$ mm	$\Delta x = 100$ mm
1 m	11.4 px	22.9 px	38.1 px
3 m	3.8 px	7.6 px	12.7 px
5 m	2.3 px	4.6 px	7.6 px

Shifts above ≈3 px are reliably perceptible in a stationary scene. At 3 m depth, the effect is clearly visible from 60 mm displacement; at 1 m, it is visible from as little as 10 mm.

9.5 Remaining Artefacts

Large disocclusions. When the head moves more than approximately 60 mm, background regions become visible with no colour data in either camera. JFA fills these with the nearest valid colour, which is plausible for small holes but visibly incorrect at high-contrast boundaries.

Textureless regions. Passive stereo fails on uniform surfaces (plain walls, overcast sky). The OAK-D confidence map is used to mask low-confidence disparity before upload; textureless areas remain as holes filled by JFA.

Iris depth variation. The fixed iris diameter constant introduces approximately ±10% depth error across the adult population, manifesting as slight over- or under-scaling of parallax magnitude.

9.6 Display Calibration

Direct calibration between the external OAK-D and the interior display is geometrically difficult: the two devices face opposite directions and are separated by the car body. A `solvePnP`-based approach chaining transforms through an intermediate reference marker was investigated but found to accumulate excessive error (1–2° angular error produces centimetre-scale positional offset at display distance). The adopted procedure initialises the display-to-camera transform from physical measurements and refines it by observing whether parallel scene lines remain parallel in the output; the dominant uncertainty sources (iris depth ±5–10%, tracker noise ±5 mm) exceed the calibration error at this stage, making a more rigorous procedure unnecessary.

10. Conclusions and Reflections

10.1 Summary

ClearVision demonstrates that a real-time, geometrically correct A-pillar transparency system is achievable on embedded GPU hardware at a cost accessible to a student project. The system achieves 25–30 FPS at 640×480 on a Jetson Nano, compared to approximately 5 FPS in the initial prototype, by offloading stereo depth to the OAK-D Lite's Myriad X VPU and implementing the render pipeline as a multi-pass OpenGL ES 3.1 compute shader chain. The virtual window projection model produces physically correct parallax with no empirical tuning constants.

Two Python proof-of-concept demonstrations — head pose vector visualisation and head-coupled off-axis projection — validated the core geometry and established design decisions before the full pipeline was built, substantially reducing implementation risk.

10.2 Key Technical Contributions

- **Virtual window projection model**: replaces the ad-hoc disparity shift approach with geometrically correct ray-plane intersection. Parallax scale is physically derived, with no tuning constants.
- **JFA hole fill**: $\mathcal{O}(\log N)$ 2D nearest-seed Voronoi fill replaces $\mathcal{O}(N)$ horizontal scan, eliminating directional streaking.
- **`floatBitsToUint` depth ordering**: enables `atomicMax` depth testing in OpenGL ES 3.1 without floating-point atomic support, exploiting the IEEE 754 float bit ordering.
- **IIR invalid-sample guard**: prevents valid depth from decaying into adjacent zero-disparity holes over time.
- **Iris-based metric depth**: uses the known iris diameter (11.7 mm) as a depth cue, avoiding a depth sensor on the driver-facing side of the system.

10.3 Limitations

Large disocclusions. JFA nearest-neighbour fill is plausible only for small holes (roughly 30 px). For larger disocclusions, background inpainting or temporal reprojection would be required. This is a fundamental constraint of single-viewpoint DIBR.

Textureless regions. Passive stereo fails on uniform surfaces. Active stereo (structured light) or a time-of-flight sensor would provide reliable depth in such scenes at substantially higher hardware cost.

Single viewer. The system is calibrated to one driver's head position. Multi-viewer support would require time-multiplexed rendering or a lenticular display.

Display calibration. The manual calibration procedure is sufficient for development but would require a repeatable mechanical fixture for production deployment.

10.4 Future Work

- **Temporal reprojection**: warp the previous frame's output into the current frame using the inter-frame head displacement, seeding holes before JFA. This would reduce visible artefacts for large disocclusions.
- **Joint bilateral depth upsampling**: upsample the 480P depth map to match the IMX214 colour resolution (up to 1080P), guided by colour edges, before the UNPROJECT pass.
- **IMU-assisted display pose**: use the OAK-D Lite's onboard Bosch BMI270 and a display-mounted IMU to automate the rotation component of the display-to-camera calibration.

10.5 Reflections

The project demonstrated that hardware-accelerated stereo depth is now accessible at consumer price points. The OAK-D Lite substantially lowered the barrier to entry: hardware SGBM, subpixel disparity, global shutter, EEPROM-stored intrinsics, and a single USB-C cable replaced what previously required a CUDA-capable GPU, a manual calibration rig, and careful CSI ribbon routing.

The majority of development time was spent not on the core DIBR rendering but on peripheral problems: build system compatibility, tracker reliability, and coordinate system consistency. These are typical of projects that integrate multiple independently developed components (DepthAI, TensorRT, OpenGL ES, Qt) on a constrained embedded platform. Documenting the specific failure modes encountered provides a concrete reference for similar work on the same hardware stack.

The virtual window model, once correctly implemented, required no further calibration. Physically correct parallax — including correct vertical parallax, depth-dependent magnitude, and natural near-object motion — made the A-pillar elimination effect substantially more convincing than the earlier disparity-shift model.

Bibliography

[1] National Highway Traffic Safety Administration, "Driver blind spot and vehicle a-pillar obstruction," Technical Report DOT HS 809 726, NHTSA, 2003.

[2] Toyota Motor Corporation, "Toyota develops technology to make A-pillars optically transparent." Toyota Global Newsroom, 2014.

[3] Continental AG, "Transparent A-pillar concept: OLED display in the pillar area." Continental Press Release, CES, 2016.

[4] Jaguar Land Rover, "Jaguar land rover develops transparent pillar and 'Follow-Me' ghost car navigation research." Jaguar Land Rover Newsroom, 2014.

[5] C. Fehn, "Depth-image-based rendering (DIBR), compression, and transmission for a new approach on 3d-tv," in *Proceedings of SPIE Stereoscopic Displays and Virtual Reality Systems XI*, vol. 5291, pp. 93–104, 2004.

[6] A. Smolic, K. Mueller, N. Stefanoski, J. Ostermann, A. Gotchev, and G. B. Akar, "Coding algorithms for 3DTV—A survey," *IEEE Transactions on Circuits and Systems for Video Technology*, vol. 17, no. 11, pp. 1606–1621, 2007.

[7] K. Müller, A. Smolic, K. Dix, P. Merkle, P. Kauff, and T. Wiegand, "View synthesis for advanced 3D video systems," *EURASIP Journal on Image and Video Processing*, vol. 2008, p. 438148, 2008.

[8] G. Rong and T.-S. Tan, "Jump flooding in GPU with applications to Voronoi diagram and distance transform," in *Proceedings of the 2006 Symposium on Interactive 3D Graphics and Games (I3D)*, pp. 109–116, 2006.

[9] B. Mildenhall, P. P. Srinivasan, M. Tancik, J. T. Barron, R. Ramamoorthi, and R. Ng, "NeRF: Representing scenes as neural radiance fields for view synthesis," in *Proceedings of the European Conference on Computer Vision (ECCV)*, pp. 405–421, 2020.

[10] D. Scharstein and R. Szeliski, "A taxonomy and evaluation of dense two-frame stereo correspondence algorithms," *International Journal of Computer Vision*, vol. 47, no. 1–3, pp. 7–42, 2002.

[11] H. Hirschmüller, "Accurate and efficient stereo processing by semi-global matching and mutual information," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, vol. 2, pp. 807–814, 2005.

[12] Luxonis Holdings Corporation, "OAK-D Lite datasheet and documentation." Luxonis Docs, 2023.

[13] P. Viola and M. J. Jones, "Robust real-time face detection," *International Journal of Computer Vision*, vol. 57, no. 2, pp. 137–154, 2004.

[14] Y. Kartynnik, A. Ablavatski, I. Grishchenko, and M. Grundmann, "Real-time facial surface geometry from monocular video on mobile GPUs," in *CVPR Workshop on Computer Vision for AR/VR*, 2019.

[15] J. P. Bergmanson and C. A. Martinez, "Size does matter: what is the corneo-limbal diameter?," *Clinical and Experimental Optometry*, vol. 100, no. 5, pp. 522–528, 2017.

[16] C. Ware, K. Arthur, and K. S. Booth, "Fish tank virtual reality," in *Proceedings of the*

INTERACT '93 and CHI '93 Conference on Human Factors in Computing Systems, pp. 37–42, 1993.

[17] J. C. Lee, "Head tracking for desktop vr displays using the wii remote." Online demonstration and code release, 2008.

[18] R. Kooima, "Generalized perspective projection," *Technical Note, School of Electrical Engineering and Computer Science, Louisiana State University*, 2009.

[19] M. Bonetto *et al.*, "Benchmarking passive stereo depth cameras under real-world conditions." arXiv preprint arXiv:2501.07421, 2025.

[20] D. S. Bolme, J. R. Beveridge, B. A. Draper, and Y. M. Lui, "Visual object tracking using adaptive correlation filters," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 2544–2550, 2010.

[21] NVIDIA Corporation, "TensorRT developer guide." NVIDIA Docs, 2024.

A. Stereo Baseline Selection

The stereo baseline determines both depth accuracy and the maximum synthesisable viewpoint shift. For the A-pillar application, both constraints converge on the same range.

The A-pillar obstruction subtends approximately 80–120 mm of lateral space at the driver's eye position (measured on a typical passenger vehicle, driver seated 600–700 mm from the pillar). The synthesised viewpoint shift must span this width. A narrower baseline would not produce sufficient shift; a wider baseline would shift near-pillar objects beyond the obstruction width, breaking the window illusion.

The OAK-D Lite's 75 mm fixed baseline lies comfortably within this range.

A further constraint: extending the baseline too far increases disocclusion region size, worsening visible artefacts from the JFA fill. For DIBR specifically, the recommended stereo baseline is approximately 5–10% of the target scene depth — for 3–5 m scenes, this gives 150–500 mm. The A-pillar geometry constraint at 80–120 mm is therefore the binding constraint, not depth accuracy.

B. Key Source File Reference

Table B.1: Principal source files

File	Role
`src/view_synthesis.cpp`	Qt OpenGL widget; compute shader dispatch; main render loop
`src/oak_receiver.cpp`	DepthAI pipeline; OAK-D polling thread
`src/face_tracker.cpp`	Haar + TRT FaceMesh pipeline; iris depth estimation
`src/face_tracker.h`	Tuning constants (`FT_SMOOTH`, iris diameter)
`src/calibration.cpp`	Legacy manual stereo calibration — retained for reference
`demos/head_coupled_offaxis.py`	Off-axis projection proof-of-concept
`demos/face_landmarker_demo.py`	Head pose vector proof-of-concept